Introduction to Operations Research, Deterministic Models

Lecture 9: Solving Integer Programming Problems

Department of Industrial Engineering, Tsinghua University

Instructor: Junyi Liu

Nov 12nd, 2024

Solving integer programming problems

- The number of solutions grows in exponential order with the number of integer variables
- Solving general integer programs can be much more difficult than solving linear programs. There is no known polynomial-time algorithm for solving general MILPs.
- For any given IP problem, the corresponding linear programming problem without integer requirements commonly is referred to as its LP relaxation.
- Solving the associated linear programming relaxation results in **an upper bound** on the optimal value to the maximization of MILP.
  - it is optimal to MILP if it satisfies the integer requirements
  - rounding to a feasible integer solution may be difficult
  - the optimal solution to the LP relaxation may be arbitrarily far away from optimality
  - rounding may result in a solution that is not feasible, or far from optimality

Branch and bound methods

Branch and bound is the most commonly-used algorithm for solving MILPs.

General form of MILP

$$\begin{array}{ll} \text{maximize} & c_1^\top x_1 + c_2^\top x_2 \\ \text{subject to} & A_1 x_1 + A_2 x_2 = b \\ & x_1 \geq 0, x_2 \geq 0 \\ & x_1 \text{ integer} \end{array}$$

Let S represent the feasible region of the above MILP problem. With a partition of S into subsets $S_1, \ldots, S_k$ such that $S = S_1 \cup \ldots S_k$. Then

$$\max_{x \in S} \quad c^\top x = \max_{i \in [k]} \left\{ \max_{x \in S_i} c^\top x \right\}$$

In other words, we can optimize over each subset separately.

• Idea: If we can't solve the original problem directly, we might be able to solve the smaller subproblems recursively.

Branching and bounding techniques

- **Branching**: dividing the original problem into subproblems based on a partition of $S = S_1 \cup \ldots \cup S_k$ with $S_i = S \cap X_i$ and $S \subseteq \bigcup_i X_i$.
  - $\{X_i\}_{i=1}^k$ is referred to an admissible disjunction valid for S if $\{X_i\}$ are polyhedron and $X_i \cap X_j = \emptyset$ for all $i, j \in [k]$.
  - We only consider the partition of polyhedrons so that the subproblem remain MILP.
  - The partition list of branching is updated along the algorithmic process
  - A typical branching of S: let x^* be an optimal solution of LP relaxation problem and select an index j whose value x_j^* is fractional. Then $S_1 = \{x \in S \mid x_j \geq \lceil x_j^* \rceil\}$ and $S_2 = \{x \in S \mid x_j \leq \lfloor x_j^* \rfloor\}$.

- **Bounding**: obtaining a bound on the optimal solution value of a subproblem
  - any feasible solution to the problem provides a lower bound $l(S)$ of the optimal value to the original MILP.
  - the optimal value of the LP relaxation problem provides an upper bound $b(S)$ of the optimal value to the original MILP.
  - for each subproblem $\max_{x \in S_i} c^\top x$, we can obtain its upper bound $b(S_i)$ by solving the corresponding LP relaxation problem

Branch and bound: initial subproblem

- We first solve the LP relaxation of the original problem with the optimal value $b(S)$, which leads to the following three cases:
  1. The LP relaxation problem is infeasible $\Longrightarrow$ MILP is infeasible.
  2. The optimal solution to the LP relaxation problem is a feasible solution for the MILP $\Longrightarrow$ optimal solution to MILP.
  3. The optimal solution to the LP relaxation problem is not feasible for the MILP $\Longrightarrow$ upper bound $b(S)$ to MILP.

- In the first two cases, we are finished.

- In the third case, we must branch and recursively solve the resulting subproblems.

Geometry of branching

Consider a feasible region defined as the set of all integer points within a polyhedron P.

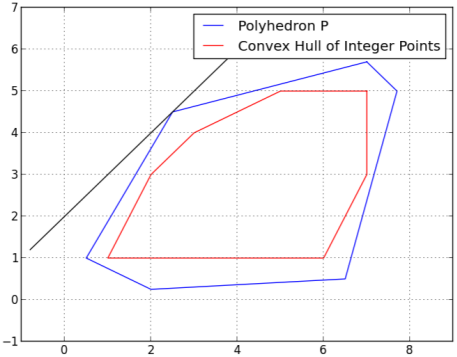


Figure: The original feasible region

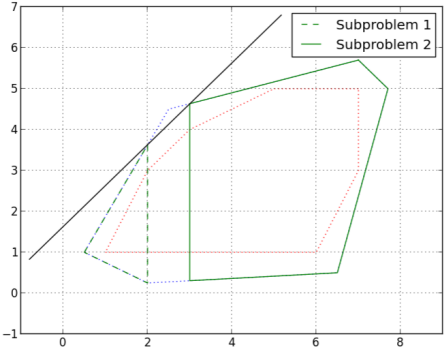


Figure: Branching on disjunction $x_1 \leq 2$ OR $x_1 \geq 3$

Branch and bound: continued

After branching, we solve each of the subproblems recursively.

• Let $v(S_i)$ denote the optimal value to the subproblem restricted to a subset S_i. We must have $v(S_i) \leq b(S_i)$.
• If $b(S_i) \leq$ the current lower bound of MILP, then we need not consider the subproblem further. This is the key to the efficiency of the algorithm.

If we picture the branching process of subproblems graphically, they form a search tree.

- **Terminology:**
  - The root node is the original MILP.
  - Each node corresponds to a subproblem, which is linked to its children via the branching process.
  - Eliminating a subproblem from further consideration is called "pruning".
  - The act of bounding and then branching is called "processing".
  - A subproblem that has not yet been considered is called "a candidate for processing.
  - The set of candidates for processing is called "the candidate list".

Branch and bound algorithm

1. To start, derive a lower bound L with the corresponding feasible solution that is referred to as the incumbent solution.
2. Put the original problem on the candidate list.
3. Select a subproblem S_i from the candidate list and solve the LP relaxation to obtain the upper bound $b(S_i)$.
   - If the LP relaxation is infeasible $\Longrightarrow$ node can be pruned.
   - If $b(S_i) \leq L \Longrightarrow$ node can be pruned.
   - If $b(S_i) > L$ and the solution is feasible for the MILP $\Longrightarrow$ set $L \leftarrow b(S_i)$ and update the incumbent solution.
   - Otherwise ($b(S_i) > L$ and the solution is infeasible), branch and add the new subproblems to the candidate list.
4. If the candidate list is nonempty, go to Step 3. Otherwise, the algorithm is completed.

The incumbent solution is the best solution on hand.

Finite convergence of BB algorithm

For LP-based branch and bound, ensuring convergence requires a convergent branching method.

• Roughly speaking, a convergent branching method is one which will guarantee that at some finite depth, any resulting bounding problem will either
- produce a feasible solution (to the original MILP); or
- be proven infeasible; or
- be pruned by bound.

• Typically, we achieve this by ensuring that at some finite depth, the feasible region of the bounding problem contains at most one feasible solution, and then we obtain the optimal solution.

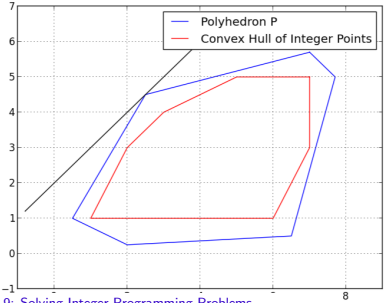
Algorithmic choices in BB algorithm

Although the basic BB algorithm is straightforward, the efficiency of it in practice depends strongly on making good algorithmic choices.

- These algorithmic choices are made largely by heuristics that guide the algorithm:
  - The bounding methods: LP relaxation corresponding to different equivalent MILP formulations, or adding cutting plane to improve the bound
  - Selecting the next candidate subproblem: "Best-first" always chooses the candidate with the highest upper bound. This rule minimizes the size of the tree.
  - Branching: choose the branching disjunction based on the predicted amount of progress it will produce towards our goal. Branching wisely is extremely important. A "poor" branching can slow the algorithm significantly.

The Importance of Formulation

- A typical MILP can have many alternative equivalent formulations.
- Each formulation corresponds to a different polyhedron enclosing the integer points that are feasible for the problem.
- Formulations may lead to different upper bounds of LP relaxation, so choosing the right one is critical in order to provide "good" upper bounds.
- The more closely the polyhedron approximates the convex hull of the integer solutions, the better the bound will be.



Example: Facility location problem

- We are given n potential facility locations and m customers.
- There is a fixed cost c_j of opening facility j.
- There is a cost d_{ij} associated with serving customer i from facility j.
- We have two sets of binary variables.
  - y_j is 1 if facility j is opened, 0 otherwise.
  - x_{ij} is 1 if customer i is served by facility j, 0 otherwise.

Example: Facility location problem

$$
\begin{array}{lll}
\text{minimize} & \displaystyle\sum_{j=1}^{n} c_j y_j + \sum_{i=1}^{m}\sum_{j=1}^{n} d_{ij} x_{ij} & \\
\text{s.t.} & \displaystyle\sum_{j=1}^{n} x_{ij} = 1, & \forall i \\
 & \displaystyle\sum_{i=1}^{m} x_{ij} \leq m y_i, & \forall j \\
 & x_{ij} \in \{0,1\}, y_j \in \{0,1\}, & \forall i,j
\end{array}
$$

$$
\begin{array}{lll}
\text{minimize} & \displaystyle\sum_{j=1}^{n} c_j y_j + \sum_{i=1}^{m}\sum_{j=1}^{n} d_{ij} x_{ij} & \\
\text{s.t.} & \displaystyle\sum_{j=1}^{n} x_{ij} = 1, & \forall i \\
 & x_{ij} \leq y_i, & \forall i,j \\
 & x_{ij} \in \{0,1\}, y_j \in \{0,1\}, & \forall i,j
\end{array}
$$

- Notice that the set of integer solutions contained in each of the polyhedra is the same.
- However, the second polyhedra is strictly included in the first one.
- The second polyhedra will yield stronger bounds and be more suitable for BB algorithm.
- Notice that the second formulation includes more constraints, but will likely solve more quickly by providing better bounds.

Reading materials

Section 12.5-12.7, *Introduction to Operations Research* by Hillier and Lieberman